

实验一：词法分析与语法分析

姓名：邵与乔 学号：231240019 日期：2026年4月3日

1. 实验目的

- 使用 **Flex** 构建 C++ 语言的词法分析器
- 使用 **Bison** 构建语法分析器，检测语法错误
- 构建**抽象语法树（AST）**，为后续语义分析和代码生成打基础
- 实现词法错误与语法错误的检测与报告

2. 实验内容

模块	文件	功能
词法分析	lexical.l	识别 token，处理空白/注释/数字/标识符/运算符
语法分析	syntax.y	根据 C++ 文法进行语法分析，构建 AST
AST	ast.h / ast.c	定义节点结构、创建、打印、释放
主程序	main 集成	调用分析器，输出 AST
错误处理	集成在 .l 和 .y 中	报告非法字符、标识符超长、数字格式错误、语法错误

3. 实现细节

3.1 词法分析器（Flex）

支持的 token 类别：

- 关键字**：int, float, struct, return, if, else, while
- 标识符**：字母/下划线开头，长度 ≤ 32 ，超长报错
- 常量**：十进制、八进制（0 开头）、十六进制（0x/0X）、浮点数（含科学计数法）
- 运算符**：=, >, <, >=, <=, ==, !=, +, -, *, /, &&, ||, !, .
- 分隔符**：;, ,, (,), [,], {, }

错误处理示例：

- 非法字符：Error type A at Line X: Mysterious character 'c'.
- 非法八进制（如 089）或十六进制（如 0x1g）同样报错

3.2 语法分析器 (Bison)

文法关键规则 (精简):

```
Program → ExtDefList
ExtDef → Specifier ExtDecList ; | Specifier ; | Specifier FunDec CompSt
Specifier → TYPE | StructSpecifier
VarDec → ID | VarDec [ INT ]
FunDec → ID ( VarList ) | ID ( )
Stmt → Exp ; | CompSt | return Exp ;
      | if ( Exp ) Stmt | if ( Exp ) Stmt else Stmt
      | while ( Exp ) Stmt
Exp → Exp = Exp | Exp && Exp | Exp || Exp | Exp RELOP Exp
    | Exp + Exp | Exp - Exp | Exp * Exp | Exp / Exp
    | ( Exp ) | - Exp | ! Exp
    | ID ( Args ) | ID ( ) | ID . ID | ID [ Exp ]
    | INT | FLOAT | ID
```

优先级与结合性 (解决二义性):

```
%right ASSIGNOP
%left OR
%left AND
%left RELOP
%left PLUS MINUS
%left STAR DIV
%right NOT UMINUS
%left DOT LB
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

3.3 抽象语法树 (AST)

节点结构:

```
typedef struct ASTNode {
    char* type;           // 如 "Exp", "Stmt", "ID"
    char* value;          // 标识符名、常量值
    int line;
    struct ASTNode* left;
    struct ASTNode* right;
    struct ASTNode* child; // 用于多子节点链
} ASTNode;
```

主要函数:

- `ASTNode* createNode(char* type, char* value, int line)`
- `void printAST(ASTNode* root, int level)`
- `void freeAST(ASTNode* root)`

3.4 错误处理

- 词法错误：在 Flex 中直接输出并继续
- 语法错误：yyerror() 报告 Error type B at Line X: syntax error.
- 错误恢复：使用 error 规则（如 error ;、error }）避免后续错误被淹没

4. 代码结构

```
task1/Lab/Code/  
├─ Makefile  
├─ lexical.l  
├─ syntax.y  
├─ ast.h / ast.c  
├─ utils.h  
└─ (生成文件: lex.yy.c, syntax.tab.c, syntax.tab.h, parser)
```

构建命令：

```
make  
./parser ../Test/test1.cmm
```

5. 实验总结

难点	解决方案
多种数字格式识别与错误检测	正则匹配 + 额外语义检查
表达式与 if-else 二义性	Bison 优先级 / %nonassoc 规则
AST 结构设计	使用 child 表示子节点链表，保留 left/right 灵活性